

## 参考文献

- [ 1 ] x86 memory segmentation [ EB/OL ]. [ 2020-04-01 ]. [https://en.wikipedia.org/wiki/X86\\_memory\\_segmentation](https://en.wikipedia.org/wiki/X86_memory_segmentation).
- [ 2 ] Intel 5-level paging [ EB/OL ]. [ 2022-05-01 ]. [https://en.wikipedia.org/wiki/Intel\\_5-level\\_paging](https://en.wikipedia.org/wiki/Intel_5-level_paging).
- [ 3 ] sel4 reference manual version 11.0.0 [ EB/OL ]. [ 2022-12-06 ]. <https://sel4.systems/Info/Docs/seL4-manual-latest.pdf>.
- [ 4 ] Fiasco.oc l4re system structure [ EB/OL ]. [ 2020-05-01 ]. [https://l4re.org/doc/l4re\\_intro.html#fiasco\\_intro](https://l4re.org/doc/l4re_intro.html#fiasco_intro).
- [ 5 ] Fuchsia virtual memory address region [ EB/OL ]. [ 2020-05-01 ]. [https://fuchsia.dev/fuchsia-src/reference/kernel\\_objects/vm\\_address\\_region](https://fuchsia.dev/fuchsia-src/reference/kernel_objects/vm_address_region).

虚拟内存管理：扫码反馈



## 物理内存管理

在内存管理方面，操作系统除了担负管理页表映射的职责外，还担负着管理物理内存资源的职责。各个应用进程在运行过程中都需要使用物理内存资源，操作系统需要为它们分配物理内存资源。一方面，操作系统需要具备以物理页为粒度进行物理内存分配的能力，从而能够在应用进程的页表中填写虚拟页到物理页的映射。另一方面，操作系统在运行的过程中会用到大量不同的结构体，许多结构体远小于一个物理页，因此也需要具备分配小内存区域的能力，否则将造成内存浪费。此外，计算平台上配备的物理内存总是有限的，在物理内存被分配完之后，操作系统可能仍会面临内存分配的需求。因此，操作系统设计了换页机制，通过把物理内存中的数据暂时换出到磁盘等次级存储上的方式释放空闲的物理内存。最后，操作系统在进行物理内存分配时，还可以结合实际的硬件特征设计可提升应用进程性能的分配策略。

本章主要包括三个方面的内容：

- 操作系统如何管理和分配物理内存页以及小内存块。
- 操作系统如何设计与实现换页机制。
- 操作系统中性能导向的物理内存分配策略。

### 5.1 操作系统的职责：管理物理内存资源

#### 本节主要知识点

- 如何评价物理内存分配器的优劣？
- 广泛使用的伙伴系统分配器的工作原理是什么？
- 什么是 SLAB 分配器？为什么有了伙伴系统之后还需要 SLAB 分配器？

#### 5.1.1 目标与评价维度

操作系统的物理内存分配设计有两个重要的评价维度。维度一，物理内存分配器要



追求更高的内存资源利用率，即尽可能减少资源浪费。这里先介绍内存碎片（Memory Fragmentation）的概念，内存碎片指的是无法被利用的内存，其直接导致内存资源利用率的下降。如何减少内存碎片是内存分配器设计者最关心的一个问题。内存碎片又被分为外部碎片（External Fragmentation）和内部碎片（Internal Fragmentation）。

如果分配器设计未能很好地考虑避免外部碎片，那么外部碎片通常会在多次分配和回收之后产生。图 5.1a 给出了外部碎片的示意图，在多次分配和回收之后，物理内存上空闲的部分处于离散分布的状态。此时，有可能出现一个内存分配请求，其请求的内存大小大于任意一个单独的空闲部分，却小于空闲部分的总和；换句话说，系统中存在足够的空闲内存，却无法满足这个请求。此时，这些无法使用的空闲物理内存被称为外部碎片。一种直观的解决外部碎片的方式是，将物理内存以固定大小（能够满足最大分配请求）划分成若干块，然后每次用一个块服务一个分配请求。如此一来，外部碎片的问题看似迎刃而解，但是又可能会导致严重的内部碎片问题。图 5.1b 给出了内部碎片的示意图。当分配的内存空间大于实际分配请求所需要的内存空间时，就会浪费部分内存，这种被浪费的内存空间即为内部碎片。

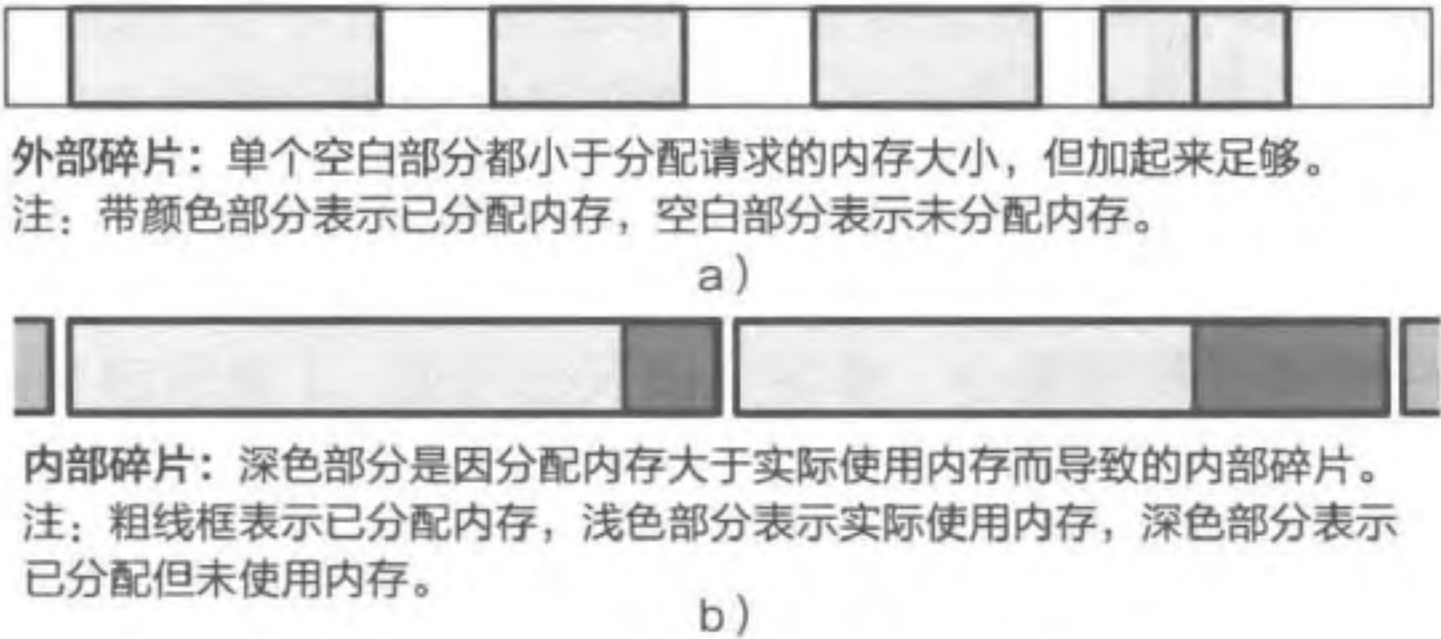


图 5.1 外部碎片和内部碎片

维度二，物理内存分配器要追求优秀的性能，主要是尽可能降低分配延迟和节约 CPU 资源。通过精密的算法细致地解决碎片问题固然能够有效提高内存资源利用率，但却可能带来高昂的性能开销，比如增加分配器完成分配请求的时间，或者由于过多后台处理而导致占用更多的 CPU 资源。

因此，一个优秀的物理内存分配器需要兼顾内存资源利用率和性能。

### 5.1.2 基于位图的连续物理页分配方法

4.2.3 节提到的 `alloc_page` 接口就是用于分配物理页的。操作系统可以通过如下方法实现物理页分配：首先初始化一个位图（Bitmap），每一位对应一个物理页，若为 0 则表明相应的物理页空闲，反之则为已分配；在分配时查找位图，找到为 0 的位，分配相应的物理页，并且把该位设置为 1。如果操作系统只需要分配单个 4KB 的物理页<sup>①</sup>，基

① 尽管 AArch64 体系结构允许操作系统选择 16KB 等作为最小页面，但本章默认 4KB 是最小物理页大小，且不影响本章对原理和实现的介绍。

于位图的简单实现已经能够满足需求。但事实上操作系统还有其他的内存分配需求：第一，分配若干连续的 4KB 物理页，例如用于 4.2.6 节提到的大页映射；第二，分配比 4KB 小的结构体变量，例如用于表示进程的结构体。基于位图的 `alloc_page()` 实现只具备分配单个 4KB 物理页的功能，对于上述两种需求而言显得捉襟见肘。

代码片段 5.1 给出了一种扩展实现<sup>⊖</sup>，支持分配连续的物理页。具体来说，操作系统仍然维护一个全局位图，每个物理页对应位图中的一位；若为 0 则表示空闲，若为 1 则表示已分配；分配  $n$  个连续的物理页只需要在位图中找到连续  $n$  位为 0 的位置，在位图中把相应的物理页标记为 1，并返回其中起始物理页的地址；释放物理页只需要在位图中将相应的位清 0 即可。

代码片段 5.1 一种简单的连续物理页分配

```

1 // 共有 N 个 4K 物理页，位图中的每一个对应一个页
2 bit bitmap[N];
3
4 void init_allocator(void)
5 {
6     int i;
7     for (i = 0; i < N; ++i)
8         bitmap[i] = 0;
9 }
10
11 // 分配 n 个连续的物理页
12 u64 alloc_pages(u64 n)
13 {
14     int i, j, find;
15
16     for (i = 0; i < N; ++i) {
17         find = 1;
18
19         // 从第 i 个物理页开始判断连续 n 个页是否空闲
20         for (j = 0; j < n; ++j) {
21             if (bitmap[i+j] != 0) {
22                 find = 0;
23                 break;
24             }
25         }
26
27         if (find) {
28             // 将找到的连续 n 个物理页标记为已分配
29             for (j = i; j < i+n; ++j)
30                 bitmap[j] = 1;
31             // 返回第 i 个物理页的起始地址
32             return FREE_MEM_START + i * 4K;
33         }

```

⊖ 为保持简洁清晰，本章示意代码省略边界条件检查。



```
34 }
35
36 // 分配失败
37 return NULL;
38 }
39
40 // 释放 n 个连续的物理页
41 void free_pages(u64 addr,u64 n)
42 {
43     int page_idx;
44     int i;
45
46     // 计算待释放的起始页索引
47     page_idx=(addr-FREE_MEM_START)/4K;
48
49     for(i=0;i<n;++i){
50         bitmap[page_idx+i]=0;
51     }
52 }
```

这种简单分配器设计在 5.1.1 节介绍的两个评价维度上均存在不足。一方面，由于该设计需要依次查询整个位图，所以分配速度慢。另一方面，该设计会导致外部碎片问题，图 5.2 展示了一个具体的例子。假设一共有 4 个空闲物理页（空闲页 0 ~ 3），在初始化时全局位图的 4 位均为 0。首先，在接收到分配 1 个页的请求后，分配器返回空闲页 0，并在位图中将第 0 位标记成 1，表示已分配。接着，在接收到分配 2 个连续页的请求后，分配器返回空闲页 1 和空闲页 2，并把位图中的第 1 位和第 2 位标记成 1。然后，在接收到释放首次分配的页的请求后，分配器在位图中将第 0 位标记成 0，表示该页处于空闲状态。最后，当再次接收到分配 2 个连续页的请求时，分配器由于无法找到满足需要的页而分配失败，但是此时实际上存在两个空闲页（空闲页 0 和空闲页 3），也就是意味着出现了外部碎片问题。简单的页分配设计容易导致外部碎片的产生，尽管物理内存容量足够，但是仍然可能出现分配失败的情况。



图 5.2 外部碎片的产生示例

### 5.1.3 伙伴系统原理

伙伴系统 (Buddy System)<sup>[1]</sup> 在现代操作系统中被广泛地用于分配连续的物理内存页，从 20 个世纪开始发展并沿用至今。其基本思想是将物理内存划分成连续的块，以块作为基本单位进行分配。不同块的大小可以不同，每个块都由一个或多个连续的物理页组成，物理页的数量必须是 2 的  $n$  次幂 ( $0 \leq n < \text{预设最大值}$ )，其中预设最大值将决定能够分配的连续物理内存区域的最大值，一般由开发者根据实际需要指定。图 5.3 展示了一个典型的伙伴系统内存组织方式。

当一个请求需要分配  $m$  个物理页时，伙伴系统将寻找一个大小合适的块，该块包含  $2^n$  个物理页，且满足  $2^{n-1} < m \leq 2^n$ 。在处理分配请求的过程中，大的块可以分裂成两半，即两个小一号的块，这两个块互为伙伴。分裂得到的块可以继续分裂，直到得到一个大小合适的块去服务相应的分配请求。在一个块被释放后，分配器会找到其伙伴块，若伙伴块也处于空闲状态，则将这两个伙伴块进行合并，形成一个大一号的空闲块，然后继续尝试向上合并。由于分裂操作和合并操作都是级联的，因此能够很好地缓解外部碎片的问题。

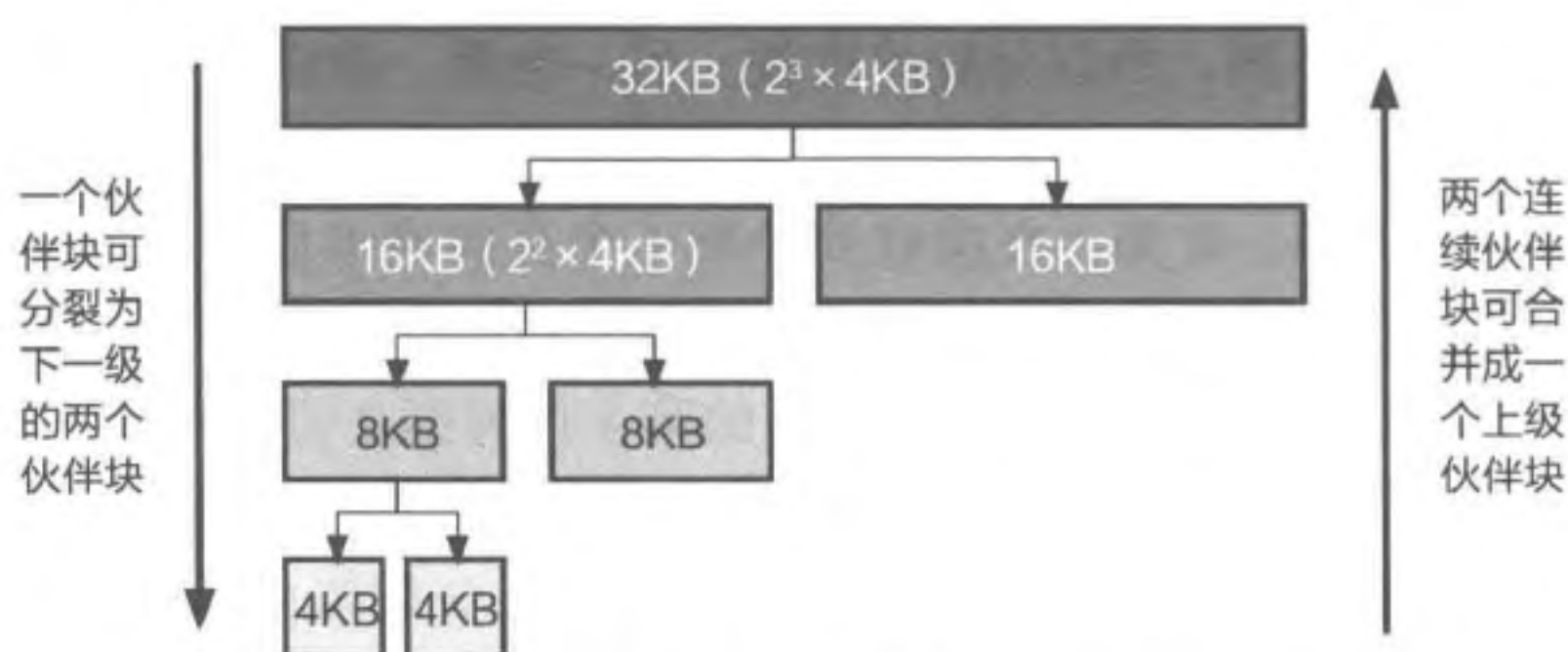


图 5.3 伙伴系统的基本思想：基于伙伴块进行分裂与合并

伙伴系统的实现通常需要用到图 5.4 所示的空闲链表数组。具体来说，全局有一个有序数组，数组中的每一项指向一个空闲链表，每条链表将其对应大小的空闲块连接起来（一个链表中的空闲块大小相同）。当接收到分配请求之后，伙伴分配器首先算出应该分配多大（大小合适）的空闲块，然后查找对应的空闲链表。在图 5.4 所示的例子中，分配请求是分配 15KB 内存，则合适的大小是 16KB，因此首先查找第 2 个 ( $2^2$ ) 空闲链表，如果链表不为空，则可以直接从链表头取出空闲块进行分配。但是，在这个例子中，该链表为空，于是分配器就会依次去存储更大块的链表中查找。由于第 3 个链表不为空，分配器就从该链表头取出空闲块 (32KB) 进行分裂操作，从而获得两个 16KB 大小的块，将其中一个用于服务请求（这里不再需要继续向下分裂），另一个依然作为空闲块插入第 2 个链表中。若再接收到一个大小为 8KB 的分配请求，分配器则会直接从第 1 个链表中取出空闲块用于服务请求。之后，若继续接收到一个大小为 4KB 的分配请求，分配



器则会取出第2个链表中大小为16KB的空闲块进行连续分裂，从而服务请求。

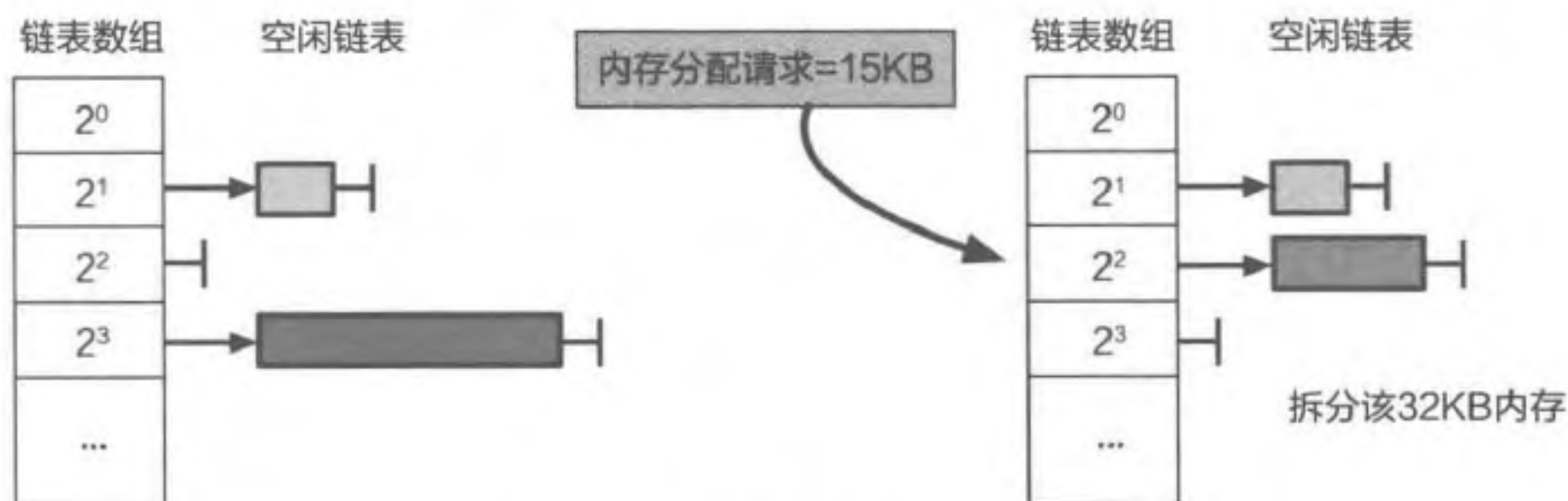


图 5.4 伙伴系统的空闲链表数组

当接收到释放块请求时，分配器首先找到待释放块的伙伴块。如果伙伴块处于非空闲状态，则将被释放的块直接插入对应大小的空闲链表中，即完成释放；如果伙伴块处于空闲状态，则将两个块进行合并，当成一个完整的块释放，并重复该过程。值得注意的是，在合并过程中寻找伙伴块的方法非常高效。互为伙伴的两个块，它们的内存地址仅有一位不同，且该位由块大小决定。所以在已知一个内存块地址的前提下，只需要翻转该地址中的一位就可以得到其伙伴块的地址，从而能够快速判断是否需要合并。举例来说，块 A ( $0 \sim 8\text{KB}$ ) 和块 B ( $8 \sim 16\text{KB}$ ) 互为伙伴块，它们的物理地址分别是  $0x0$  和  $0x2000$ ，根据 4.2.1 节介绍的直接映射方式，它们在内核地址空间中的虚拟地址分别为固定偏移  $+0x0$  和固定偏移  $+0x2000$ ，仅有第 13 位不同，而块大小是  $8\text{KB}$  ( $2^{13}\text{KB}$ )。

## 练习

- 5.1 操作系统为什么需要能够分配连续的物理页呢？
- 5.2 从图 5.4 初始的空闲链表数组中，依次分配 15KB、8KB、4KB 三次内存后，请问空闲链表  $0 \sim 4$  各剩余几个空闲块？

### 5.1.4 案例分析：ChCore 中伙伴系统的实现

代码片段 5.2 给出了表示物理页和伙伴系统空闲链表的结构体。操作系统会创建 `physical_page` 结构体的数组，其中每个结构体对应一个物理页。如代码片段 5.3 所示，伙伴系统的初始化函数 `init_buddy` 首先对该数组进行初始化，即依次初始化数组中的每个 `physical_page` 结构体。值得注意的是，每个结构体会被标记已分配（`allocated` 设置为 1）且所属伙伴块仅有一个物理页（`order` 设置为 0）。之后，`init_buddy` 函数将利用物理页释放接口依次释放每个物理页（函数中最后一个 `for` 循环），在释放每个物理页的过程中，空闲的伙伴块会被级联地合并，并且插入伙伴系统相应的空闲链表中。

代码片段 5.2 物理页结构体和伙伴系统空闲链表数组的示意代码

---

```

1 struct physical_page {
2     // 是否已经分配
3     int allocated;
4     // 所属伙伴块大小的幂次
5     int order;
6     // 用于维护空闲链表, 把该页放入 / 移出空闲链表时使用
7     list_node node;
8 };
9
10 // 伙伴系统的空闲链表数组
11 list free_lists[BUDDY_MAX_ORDER];

```

---

代码片段 5.3 伙伴系统初始化

---

```

1 // 伙伴系统初始化
2 void init_buddy(struct physical_page *start_page,
3                 u64 page_num)
4 {
5     int order;
6     int index;
7     struct physical_page *page;
8
9     // 初始化物理页结构体数组
10    for (index = 0; index < page_num; ++index) {
11        page = start_page + index;
12        // 标记成已分配
13        page->allocated = 1;
14        page->order = 0;
15    }
16
17    // 初始化伙伴系统的各空闲链表
18    for (order = 0; order < BUDDY_MAX_ORDER; ++order) {
19        init_list(&(free_lists[order]));
20    }
21
22    // 通过释放物理页的接口把物理页插入伙伴系统的空闲链表
23    for (index = 0; index < page_num; ++index) {
24        page = start_page + index;
25        buddy_free_pages(page);
26    }
27 }

```

---

代码片段 5.4 展示了 `buddy_alloc_pages` 函数和 `buddy_free_pages` 函数的逻辑, `buddy_alloc_pages` 接收 `order` 参数, 用于分配  $2^{\text{order}}$  个连续物理页。它首先把参数 `order` 作为索引, 在伙伴系统的空闲链表数组中定位到对应的空闲链表。若该链表非空 (块大小为  $2^{\text{order}}$  个物理页), 则从该链表中取出一个空闲块返回即可。否则, 依次查找下一个空闲链表, 即寻找更大的空闲块。由于取出的空闲块可能大于所需大小,



故需要调用 `split_page` 函数，该函数会将空闲块级联地分裂到所需大小。例如，若找到的空闲块大小为 8 个物理页，而需要分配 2 个物理页，则 `split_page` 会把该块拆分成一个包含 4 个物理页的空闲块和两个包含 2 个物理页的空闲块，最后分配其中一个包含 2 个物理页的空闲块，剩余两个空闲块均插入相应大小的空闲链表中。值得注意的是，`buddy_alloc_pages` 仅把分配块的第一个物理页标记成已分配，而不修改其余页对应的 `physical_page` 结构体。这能够减少内存修改操作从而提升性能，而这么做可行的原因是这些页在被分配后不位于伙伴系统的任一空闲链表中（因此不会被再次分配）。`buddy_free_pages` 用于释放块，它在标记待释放块为空闲后，调用 `merge_page` 级联地合并空闲伙伴块，然后把合并后的更大的空闲块放入相应大小的空闲链表中。

代码片段 5.4 伙伴系统的分配与释放函数

```

1 // 分配伙伴块：2^order 数量的连续 4K 物理页
2 struct page *buddy_alloc_pages(u64 order)
3 {
4     int cur_order;
5     struct list_head *free_list;
6     struct page *page = NULL;
7
8     // 搜寻伙伴系统中的各空闲链表
9     for (cur_order = order; cur_order < BUDDY_MAX_ORDER; ++cur_order) {
10         free_list = &(free_lists[cur_order]);
11         if (!list_empty(free_list)) {
12             // 从空闲链表中取出一个伙伴块
13             page = get_one_entry(free_list);
14             break;
15         }
16     }
17
18     // 若取出的伙伴块大于所需大小，则进行分裂
19     page = split_page(order, page);
20     // 标记已分配。示意代码忽略分配失败的情况
21     page->allocated = 1;
22     return page;
23 }
24
25 // 释放伙伴块
26 void buddy_free_pages(struct page *page)
27 {
28     int order;
29     struct list_head *free_list;
30
31     // 标记成空闲
32     page->allocated = 0;
33     // (尝试) 合并伙伴块
34     page = merge_page(page);
35

```

```
36 // 把合并后的伙伴块放入对应大小的空闲链表
37 order = page->order;
38 free_list = &(free_lists[order]);
39 add_one_entry(free_list, page);
40 }
```

让我们回顾 5.1.2 节给出的内存分配的例子，分析伙伴系统分配内存的效果。如图 5.5 所示，伙伴系统的初始化函数 `init_buddy` 调用 `buddy_free_pages` 依次释放每个物理页，在初始化完成后，由于级联合并操作而形成一个包含 4 个物理页的空闲块。当分配 1 个物理页时，`buddy_alloc_pages` 函数会对上述空闲块进行级联拆分，最终在空闲链表中留下一个包含 2 个物理页的空闲块和一个包含 1 个物理页的空闲块。接下来，当收到分配 2 个连续页的请求时，伙伴系统返回刚拆分出的包含 2 个物理页的空闲块。然后，在接收到释放首次分配的页的请求后，伙伴系统会把两个互为伙伴的单页空闲块合并成一个包含 2 个物理页的空闲块。最后，当再次接收到分配 2 个连续页的请求时，伙伴系统返回刚合并的空闲块即可，因没有造成外部碎片而成功满足分配请求。

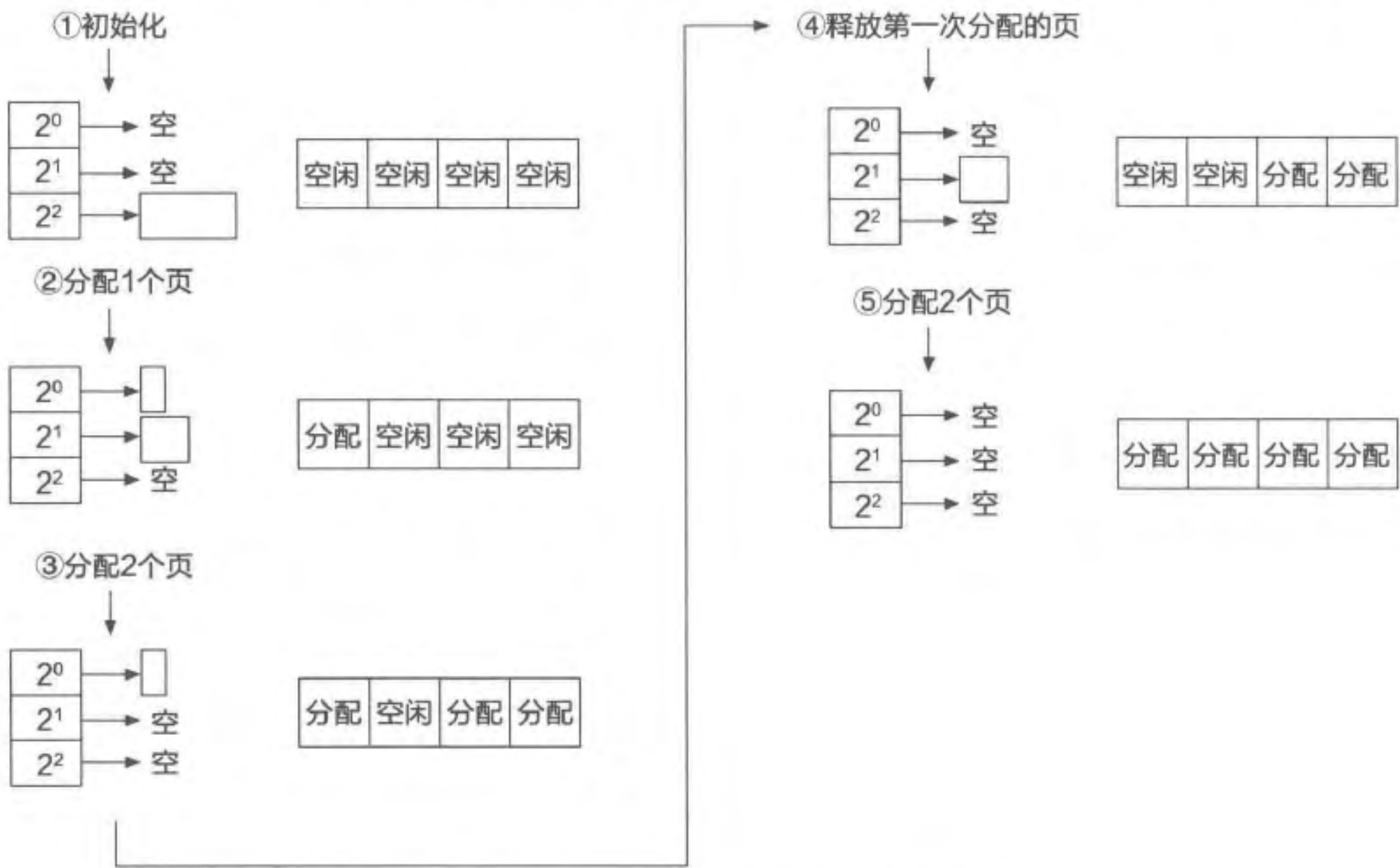


图 5.5 伙伴系统避免外部碎片的例子

练习

5.3 伙伴系统中的分配操作在最理想和最坏情况下的时间复杂度各是多少？



### 5.1.5 SLAB 分配器的基本设计

伙伴系统最小的分配单位是一个物理页（4KB），但是大多数情况下，内核需要分配的内存大小通常是几十字节或几百字节，远远小于一个物理页的大小。如果仅使用伙伴系统进行内存分配，会出现严重的内部碎片问题，从而导致内存资源利用率降低。比如，当操作系统需要分配代码片段 4.7 中所示的 `process` 和 `vmregion` 结构体时，分配单个 4KB 页作为一个结构体会浪费很多内存，因为结构体的大小只有几十字节。于是，操作系统开发人员设计了另外一套内存分配机制（SLAB 分配器），用于在操作系统中分配小内存。

#### 小知识：SLAB 小历史

20 世纪 90 年代，Jeff Bonwick 最先在 Solaris 2.4 操作系统内核中设计并实现了 SLAB 分配器<sup>[2]</sup>。之后，该分配器被 Linux、FreeBSD 等操作系统广泛地使用并得以发展。21 世纪初，操作系统开发人员逐渐发现 SLAB 分配器存在一些问题，比如维护了太多的队列，实现日趋复杂，存储开销也由于复杂的设计而增大等。于是，开发人员在 SLAB 分配器的基础上设计了 SLUB 分配器<sup>[3]</sup>。SLUB 分配器极大简化了 SLAB 分配器的设计和数据结构，降低复杂度的同时依然能够提供与原来相当甚至更好的性能，同时也继承了 SLAB 分配器的接口。此外，SLAB 分配器家族中还有一种最简单的分配器，称为 SLOB 分配器<sup>[4]</sup>，它的出现主要是为了满足内存资源稀缺场景（比如嵌入式设备）的需求，它具有最小的存储开销，但在碎片问题的处理方面比不上其他两种分配器。SLAB、SLUB、SLOB 三种分配器往往被统称为 SLAB 分配器，从 Linux-2.6.23 之后，SLUB 分配器成为 Linux 内核默认使用的分配器。

本节介绍一种常用的 SLAB 分配器（SLUB）的基本设计思路。首先，SLUB 分配器是为了满足操作系统（频繁的）分配小对象的需求，其依赖于伙伴系统进行物理页的分配。简单来说，SLUB 分配器做的事情是把从伙伴系统分配的大块内存进一步细分成小块内存进行管理。一方面由于操作系统频繁分配的对象大小相对比较固定，另一方面为了避免外部碎片问题，所以 SLUB 分配器只分配固定大小的内存块，块大小通常是  $2^n$  字节（一般来说， $3 \leq n < 12$ ）。在具体实现过程中，程序员可以根据实际需要设置其他大小来减少内部碎片。对于每一种块大小，SLUB 分配器都会使用独立的内存资源池进行分配。

图 5.6 描述的是 SLUB 分配器为分配某一种大小的内存块所维护的内存资源池。SLUB 分配器向伙伴系统申请一定大小的物理内存块（一个或多个连续的物理页），并将获得的物理内存块作为一个 slab。slab 会被划分成等长的小块内存，并且将其内部空闲的小块内存组织成空闲链表的形式。一个内存资源池通常还有 `current` 和 `partial` 两个指针。`current` 指针仅指向一个 slab，所有的分配请求都将从该指针指向的 slab 中获得空闲内存块。`partial` 指针指向由所有拥有空闲块的 slab 组成的链表。



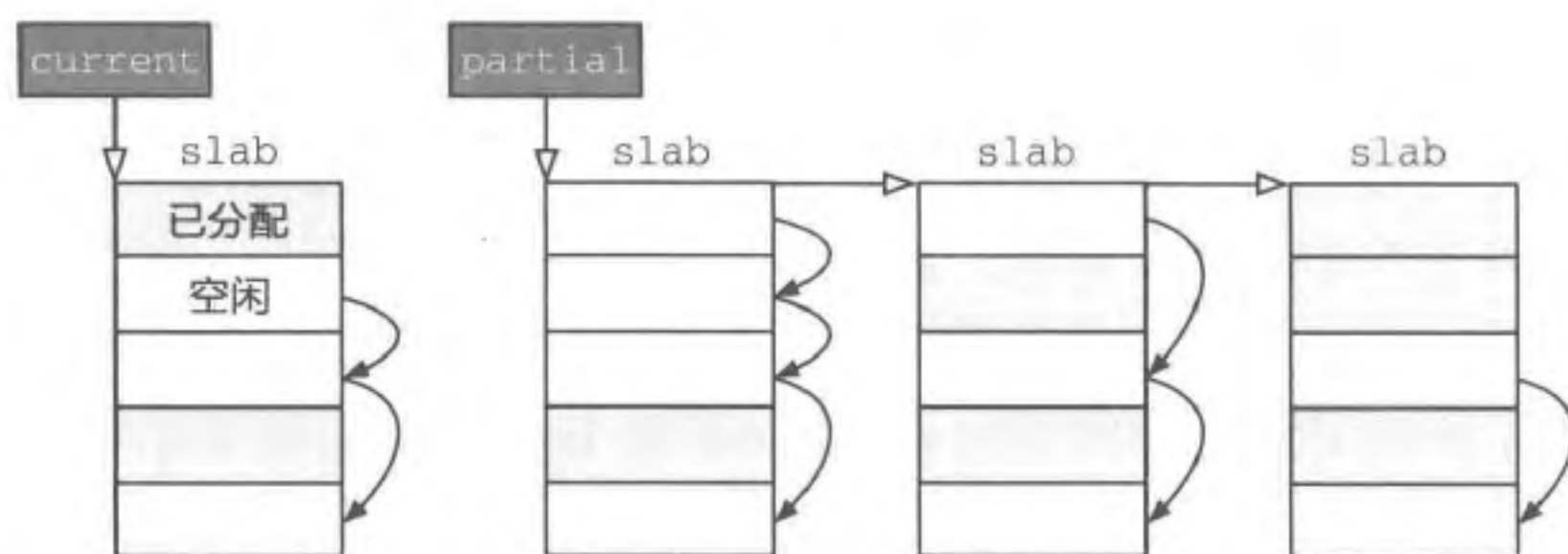


图 5.6 SLUB 分配器主要数据结构示意图。图中每个 slab 中的空白部分表示空闲，非空白部分表示已被分配

当 SLUB 分配器接收到一个分配请求时，它首先定位到能满足请求大小且最接近的内存资源池，然后从 `current` 指针指向的 slab 中取出一个空闲的块返回。如果 `current` 指针指向的 slab 在取出一个空闲块后，该 slab 不再拥有空闲块，即全部分配完，则从 `partial` 指针指向的链表中取出一个 slab 交给 `current` 指针。如果 `partial` 指针指向的链表为空，那么 SLUB 分配器就会向伙伴系统申请分配新的物理内存作为新的 slab。这样的分配设计一方面有效避免了外部碎片，另一方面通常分配速度很快（直接从 `current` 指针指向的 slab 中取出第一个空闲块即可）。通过合理地设置不同大小的内存资源池，也能够尽可能减小内部碎片导致的开销。

当 SLUB 分配器接收到一个释放请求时，它将被释放的块放入相应 slab 的空闲链表中。如果该 slab 原本已经没有空闲块，即全部分配完，则将其重新移动到 `partial` 指针指向的链表中；如果该 slab 变为所有内存块都是空闲的，即原来仅分配出去一块，那么可以将其释放并还给伙伴系统。至于如何找到释放块所属的 slab，则可以通过在 slab 头部加入元数据并且使得 slab 头部具有对齐属性等方式实现。

这里介绍的仅是 SLUB 分配器的基本设计思想。随着计算机体系结构的不断发展，今天的 SLUB 机制实际上有更多为体系结构而做的优化，比如针对多核、NUMA（Non Uniform Memory Access）架构的本地性设计等，5.3 节将介绍部分演进机制。

### 小知识：kmalloc 与 malloc

Linux 操作系统中的内存分配接口称为 `kmalloc`，其实现主要采用伙伴系统和 SLAB 分配器的设计。ChCore 在内核态也实现了类似的 `kmalloc` 接口，当 `kmalloc` 分配小于 4K 页大小的内存时，其实际上调用 SLAB 分配器；当分配大于 4K 页大小的内存时，其实际上调用伙伴系统分配若干连续的物理页。

4.2.5 节提到应用进程会使用 `malloc` 接口分配内存并用于创建变量。`malloc` 有多种实现方式，比如可以采用 SLAB 分配器的思路，或者采用 5.1.6 节介绍的空闲链表。`malloc` 在堆（进程的一个虚拟内存区域）中进行虚拟内存分配，而操作系统在不需应用感知的情况下利用伙伴系统为该虚拟内存区域分配物理页并填写页表映射。



练习

5.4 是否需要再维护一个 full 指针，用于指向全满 slab 组成的链表呢？

5.1.6 常用的空闲链表

除了上述伙伴系统和 SLAB 分配器之外，还有其他基于不同空闲链表的内存分配方法。这些方法不仅可以用在内核态，也可以用在用户态的内存分配器中，比如堆分配器。本节主要介绍三种常见的空闲列表：隐式空闲列表、显式空闲列表和分离空闲列表。

第一种简单的空闲链表称为隐式空闲链表（Implicit Free List）。如图 5.7a 所示，链表里的每个元素代表一块内存区域，空闲（空白块）和非空闲的内存块混杂在同一个链表里。每个内存块头部存储了关于该块是否空闲的信息以及块大小信息。通过块大小，可以找到下一个块的位置。在分配空闲块的时候，分配器在这个链表中依次查询，找到第一块大小足够的空闲内存块即可返回。如果找到的空闲块大小不仅能够满足分配请求，还有足量剩余，则将该块进行分裂，一部分用于服务请求，剩余部分留用为新的空闲块，从而缓解内存碎片问题。在释放内存块的时候，为了尽可能避免外部碎片问题，分配器会检查该内存块紧邻的前后（根据块地址）两个内存块是否空闲，如果有空闲块存在，则进行合并，产生更大的空闲块。

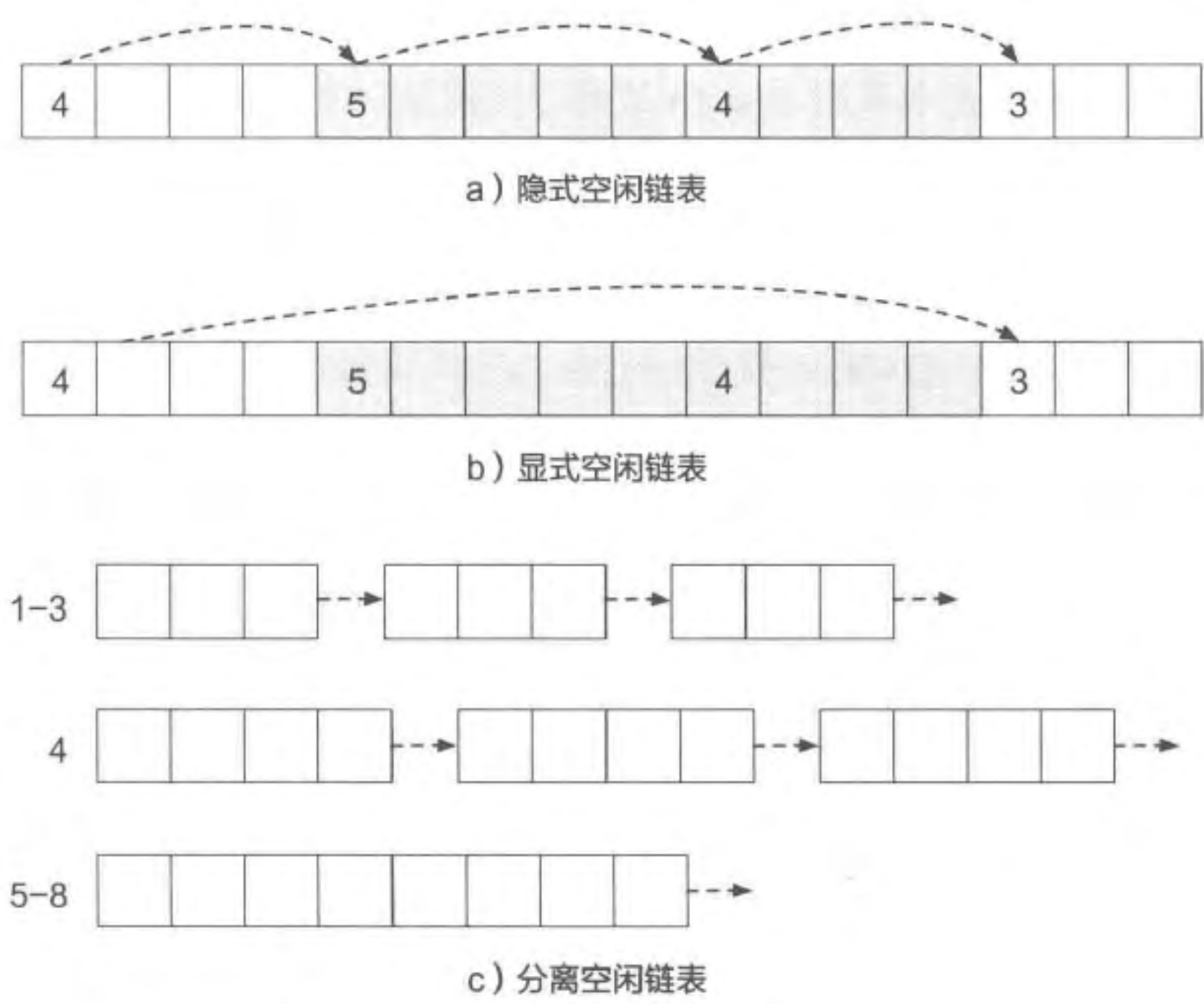


图 5.7 内存分配器中常用的三种空闲链表

与之很相似的另一种空闲链表称为显式空闲链表 (Explicit Free List)，如图 5.7b 所示。与隐式空闲链表最大的区别在于，显式空闲链表仅把空闲的内存块（而不是所有内存块）放在链表中。由于下一个空闲内存块可能存在内存中的任何位置，每个空闲块不能再依靠块大小找到下一个空闲块的位置，所以，除了块大小（合并块需要），每个空闲块需要额外维护两个指针（prev 和 next）来指向前后空闲块。不过，分配器仅需要在空闲块中维护指针，因此可以直接在空闲块的数据部分记录上述两个指针，无须使用额外的空间记录链表指针。显式空闲链表中内存块分配和释放的过程与隐式空闲链表中类似，不再赘述。相比之下，显式空闲链表在分配速度上具有优势，因为它的分配时间仅与空闲块数量成正相关，而隐式空闲链表的分配时间与所有块的数量成正相关。这个优势在内存使用率高的情况下更加明显，因为空闲块的数量更少而非空闲块更多。

另一种空闲链表数据结构称为分离空闲链表 (Segregated Free List)，是在显式空闲链表的基础上构建的。如图 5.7c 所示，其基本思想是维护多个不同的显式空闲链表，每个链表服务固定范围的分配请求，这一点和之前介绍的伙伴系统或 SLAB 分配器相似。分配内存块时，首先找到块大小对应的显式空闲链表，从中取出一个空闲块，若满足分配大小后有剩余，则将剩余部分插入相应大小的空闲链表中。如果在块大小对应的显式空闲链表中找不到合适的空闲块，则依次去更大的块大小对应的链表中寻找。释放块时，分配器依然可以先采用单个显式空闲链表中的合并策略，然后将合并产生的空闲块插入对应大小的空闲链表中。相比于普通显式空闲链表，分离空闲链表的优势在于能够获得更好的性能，一方面分配空闲块所需要的时间一般会更短，另一方面多个空闲链表可以更好地支持并发操作。此外，在分离空闲链表中采用 first-fit 策略（找到第一个空闲块即返回）能够近似地达到 best-fit 策略（最优策略，即找到大小最接近的空闲块）的内存利用率。如果分离空闲链表为每一种块大小都分配一个空闲链表，那么采用 first-fit 策略实际上也就是 best-fit 策略。

## 5.2 操作系统如何获得更多物理内存资源

### 本节主要知识点

- 物理内存不够用时怎么办？什么是换页机制？
- 如何选择页面进行替换？页替换的策略有哪些？
- 如何利用虚拟内存抽象节约物理内存（内存去重和内存压缩）？

### 5.2.1 换页机制

当操作系统需要为应用进程分配物理页时，若已经没有剩余的空闲物理页了，操作系统会束手无策吗？答案是否定的。假设如下场景：小明的笔记本配有 4GB 的物理内存，他一边使用 Photoshop 编辑大量高清图片——共需要占用 2GB 物理内存，一边玩英